

High-Performance Accelerator for Constant-Time Cross-Domain Integer and Montgomery Inversion on FPGA

XIAOYU JI, Shandong University, Qingdao, China

CHENG CHEN, Shandong University, Qingdao, China

GANGQIANG YANG, Shandong University, Qingdao, China

HONGCHAO ZHOU, Shandong University, Qingdao, China

HAILIANG XIONG, Shandong University, Qingdao, China

XIANYE BEN, Shandong University, Qingdao, China

ZHIGUO WAN, Zhejiang Lab, Hangzhou, China

Modular Inversion (MI) is one of the fundamental arithmetic operations in the finite field, which plays an essential role in various cryptographic applications and requires high performance and security. Unfortunately, the simple MI algorithm is vulnerable to side-channel attacks, such as the timing attack, which can compromise the cryptographic system by analyzing the time taken to execute cryptographic algorithms. Attackers may recover the initial data since the time can differ based on the input. Besides, the low complexity and low resource consumption of hardware implementations in MI are also challenging. In this article, we propose two novel modular inversion algorithms, named Constant-Time Integer Modular Inversion (CT-IMI) and Constant-Time Complementary Montgomery Modular Inversion (CT-CMMI). They both consist of constant iteration rounds to resist the timing attack. CT-IMI processes the data in the integer field, which is designed for common scenarios. CT-CMMI is suitable for the cross-domain case, which can directly use data in the Montgomery domain and avoid the conversion steps for some specific applications, e.g., scalar multiplication in Elliptic Curve Cryptography (ECC). In software simulations, we measure the average clock cycles for a single inversion and illustrate the relationship between various bit lengths and the latency. The significant differences between constant and non-constant algorithms demonstrate the vulnerability of modular inversion to timing attacks. In addition, we design two efficient hardware architectures on FPGA. Experimental results show that our CT-IMI can finish a single inversion in 2.56 μ s with 4.2k LUTs, 1.8k FFs, and our CT-CMMI requires 2.45 μ s with 2.7k LUTs, 1.6k FFs. The product of area and latency of our CT-IMI and CT-CMMI can

Xiaoyu Ji and Cheng Chen have equal contributions to this article.

This work was supported by the Natural Science Foundation of Shandong Province (ZR2022QF033, ZR2022LZH005), in part by the Major Basic Research Program of the Shandong Provincial Natural Science Foundation (ZR2020ZD01), in part by the Natural Science Foundation of China (62322111, 62271289), in part by the Natural Science Fund for Outstanding Young Scholars of Shandong Province (ZR2022YQ60), in part by the Research Fund for the Taishan Scholar Project of Shandong Province (tsqn202306064), and in part by Jinan “20 Terms of New Universities” Funding Project (202333035).

Authors’ Contact Information: Xiaoyu Ji, Shandong University, Qingdao, China; e-mail: 201900800217@mail.sdu.edu.cn; Cheng Chen, Shandong University, Qingdao, China; e-mail: ccsdu17@mail.sdu.edu.cn; Gangqiang Yang (corresponding author), Shandong University, Qingdao, China; e-mail: g37yang@sdu.edu.cn; Hongchao Zhou (corresponding author), Shandong University, Qingdao, China; e-mail: hongchao@sdu.edu.cn; Hailiang Xiong, Shandong University, Qingdao, China; e-mail: hailiangxiong@sdu.edu.cn; Xianye Ben, Shandong University, Qingdao, China; e-mail: benxianye@gmail.com; Zhiguo Wan, Zhejiang Lab, Hangzhou, China; e-mail: wanzhiguo@zhejianglab.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1539-9087/2026/01-ART1

<https://doi.org/10.1145/3777365>

reach 10.50 and 6.62, respectively, which shows optimal performance compared with all the results in the existing literature.

CCS Concepts: • **Security and privacy** → **Public key (asymmetric) techniques**; • **Mathematics of computing** → **Finite Field Algorithm**; • **Hardware** → **Reconfigurable logic and FPGAs**;

Additional Key Words and Phrases: Finite field arithmetic, modular inversion, montgomery algorithm, elliptic curve cryptography, side-channel attack, FPGA

ACM Reference Format:

Xiaoyu Ji, Cheng Chen, Gangqiang Yang, Hongchao Zhou, Hailiang Xiong, Xianye Ben, and Zhiguo Wan. 2026. High-Performance Accelerator for Constant-Time Cross-Domain Integer and Montgomery Inversion on FPGA. *ACM Trans. Embedd. Comput. Syst.* 25, 1, Article 1 (January 2026), 26 pages. <https://doi.org/10.1145/3777365>

1 Introduction

Modular inversion is one of the modular arithmetic used in public-key cryptosystems, such as **Elliptic Curve Cryptography (ECC)** [16, 23], **Diffie-Hellman (DH)** protocols [9], and so on. Compared with other modular algorithms, i.e., modular addition or multiplication, inversion is too expensive in implementation and should be avoided as much as possible. Although tricks on the representation of coordinates can reduce the number of modular inversions, it is still irreplaceable because the result must be converted back to affine coordinates.

However, commonly used modular inversion algorithms pose significant risks when implemented in hardware. These algorithms share a common characteristic: they contain input-dependent conditional branches. This feature directly leads to the generation of variable information during execution, such as execution time. It is a great threat to the entire cryptosystem. For example, in ECC/ECDSA cryptosystems, modular inversion is usually used to convert coordinates from projective to affine. If we adopt the **Binary Extended Euclidean Algorithm (BEEA)** to calculate the inversion, the execution flow is heavily reliant on its inputs, which means the time required to get the final result is also dependent on the inputs. This attribute is investigated in an attempt to find side-channel vulnerabilities and can seriously affect the privacy of secret variables processed by BEEA. An adversary could obtain the secret key based on the disclosure of information.

The aforementioned scenario involves a commonly used attack method known as a timing attack, which are a subset of **Side-Channel Attacks (SCAs)**. There are multiple SCAs targeting implementations of modular inversion, such as **Simple Power Analysis (SPA)** [33], **Cache-Timing Attack (CTA)** [20], **Machine Learning-based Profiling Attack (MLPA)** [32]. However, timing attacks are especially powerful against modular inversion implementations. The inherently high computational cost of modular inversion distinguishes it from other operations in terms of execution time. The dependency between outputs and inputs in modular inversion algorithms also enables attackers to deduce secret information by analyzing time. Furthermore, side-channel resistant implementations need additional countermeasures, which will introduce additional processes and increase the overhead in hardware. Thus, considering both efficiency and security, we focus on protecting modular inversion algorithms against timing attacks in our article.

One of the most important countermeasures is to ensure that the implementation has constant run-time (worst case), implying that the execution time is unaffected by the input. Constant time algorithms can resist the most basic timing attacks and are necessary for scenarios involving sensitive information. Hence, the main idea of this work is to compute an inverse using the same number of constant-time iterations each time. In our article, we propose two novel modular inversion algorithms over $\mathbb{F}_p$, which can both prevent side-channel information leakage and have high computation efficiency. We also design two efficient hardware architectures based on our

algorithms, named **Constant-Time Integer Modular Inversion (CT-IMI)** and **Constant-Time Complementary Montgomery Modular Inversion (CT-CMMI)**, respectively. CT-IMI mainly processes the number in the same field as classic algorithms, while CT-CMMI performs a cross-domain modular inversion. They show optimal performance compared with all existing work, which proves that our algorithms are effective. In general, our designs have the following features:

- High Security. Our modular inversion algorithms have a constant time feature to resist timing attacks.
- High Efficiency. Different from **Extend Euclid Algorithm (EEA)** and **Montgomery Inversion (MI)**, our designs not only have good performance in software but also are friendly to hardware. We use subtraction and shift operations instead of divisions and multiplications, which consumes less hardware resources compared to EEA and MI.
- Cross-Domain. The classic modular inversion algorithm processes integers in the same field. But in our cross-domain designs, inputs are in the Montgomery domain and the results are in the integer domain, which can avoid the extra conversion step.

The remainder of this article is organized as follows. Sections 2 and 3 provide the existing algorithms about modular inversion and introduce state-of-the-art works. Section 4 proposes our novel algorithms and theoretically analyzes their principles. Section 5 shows our hardware architectures and introduces the details of hardware implementations. The comparison results in software and hardware are listed in Section 6. Finally, Section 7 summarizes our work.

2 Background

In this section, we briefly explain some basic concepts and classic applications of modular inversion firstly. Then we introduce the Montgomery domain and the relationship between the Montgomery domain and the integer domain. Finally, we elaborate on **Fermat's Little Theorem (FLT)**, **Extended Euclidean Algorithm (EEA)** and MI algorithm as they are the basis for almost all research on modular inversion algorithms today.

2.1 Basic Concepts and Classic Applications of Modular Inversion

Modular inversion is used to finish the division operation in a finite field. For an integer a , the modular inversion of a in field p can be represented as a^{-1} . It satisfies $a \cdot a^{-1} \bmod p = 1$, where p is a prime and $a < p$.

In public key cryptography, modular inversion is a common operation in finite fields. It needs to perform multiple iteration rounds to find the results, which causes a lot of logic resource costs and long time delay on hardware compared to other arithmetic (including addition, subtraction, and even multiplication). So many works tried to avoid the inversion as much as possible and get a low-cost solution. For example, **Point Addition (PA)** and **Point Doubling (PD)** are two basic operations in Elliptic Curve Encryption. Suppose that there are two points $A(x_a, y_a)$ and $B(x_b, y_b)$ on an elliptic curve $y^2 = x^3 + ax + b$. We assume $C(x_c, y_c)$ as the result of point addition $A + B$. The coordinates of C can be calculated by the following equations:

$$\begin{aligned} x_c &= \lambda^2 - x_a - x_b \\ y_c &= \lambda(x_a - x_c) - y_a \end{aligned} \quad (1)$$

where $\lambda = \frac{y_b - y_a}{x_b - x_a}$. Obviously, one modular inversion is required according to the expression of λ . Point doubling is the same.

If we repeat them many times, the hardware resource consumption and latency are unbearable. A common solution is using some special coordinate system, such as the Jacobian projective coordinate system. Modular inversion is replaced by some modular multiplications in this system.

However, the final result still requires an inversion, which can take them back to the form of affine coordinates. For example, for a point (X, Y, Z) in the Jacobian projective coordinate system, its corresponding point in the affine coordinate system is $(\frac{X}{Z^2}, \frac{Y}{Z^3})$. Modular inversion is necessary here.

Another issue worth considering is whether modular inversion should be considered as a dedicated module. Several commonly used algorithms often use multiplications instead of divisions, such as FLT and Montgomery Modular Inversion, and so on. However, modular inversion based on these methods heavily relies on multipliers, resulting in slow speed. It also places greater pressure on the design of multipliers. Additionally, implementing a multiplier generally requires a large amount of on-chip resources such as DSPs on FPGAs. The resource consumption is not optimistic. So many works have begun to design and optimize modular inversion as a dedicated unit. For example, SASDRICH etc. [29] implemented a dedicated inversion unit based on the BEEA in their design of Curve25519 ECC cryptography system. The computation of the inversion is significantly faster and the hardware costs can be mitigated in a multi-core scenario. Thus, it is quite reasonable and necessary to design and optimize the modular inversion as a separate module.

2.2 Montgomery Domain

For an integer a and a finite field prime p , we can make a conversion such that $\tilde{a} = a \cdot 2^l \bmod p$, where l is an integer and $2^l > p$. $\tilde{a}$ is the form of a in the Montgomery domain. Many arithmetic operations in this domain are more convenient, especially Montgomery modular reduction and Montgomery modular multiplication [24]. We can also combine simple multiplication and modular reduction to finish modular multiplication and avoid using the Montgomery domain. For some Mersenne or pseudo-Mersenne primes, there exist some optimization algorithms [7, 31] for high-performance modular reduction. In summary, Montgomery's methods are universal and perform significantly superior to ordinary modular multiplication when processing continuous multiplications.

We denote Montgomery multiplication as **MontMul**($\cdot$), and Montgomery reduction as **MontRedc**($\cdot$). For integer a, b and their corresponding forms in the Montgomery domain $\tilde{a} = a \cdot 2^l \bmod p, \tilde{b} = b \cdot 2^l \bmod p$, the following equations are satisfied:

$$\mathbf{MontRedc}(a) = a \cdot 2^{-l} \bmod p, \quad (2)$$

$$\mathbf{MontMul}(a, b) = \frac{a \cdot 2^l \cdot b \cdot 2^l}{2^l} \bmod p = a \cdot b \cdot 2^l \bmod p. \quad (3)$$

Assume an integer $c = a \cdot 2^{-l} \bmod p$, then we can use **MontRedc**(a) to get c . This trick plays an important role in the design of CT-CMMI.

2.3 Basic Modular Inversion Algorithms

We select three basic modular inversion algorithms here, i.e., FLT, EEA, and MI. Other algorithms stem from them.

FLT (Algorithm 1) is a special case of the Euler Theorem. It says that $a^{p-1} \equiv 1 \bmod p$ when a is an integer and p is a prime. Since $a^{p-2} \bmod p \equiv a^{-1} \bmod p$, the modular inversion can be solved. It requires many times of multiplication and squaring, which limits its efficiency.

A tradeoff between the number of iteration rounds and the efficiency of each iteration is the key to modular inversion. EEA (Algorithm 2) has a small number of iteration rounds, but time-consuming multiplication and division operations are required in each iteration, which consumes more hardware resources and is not suitable for resource-constrained devices. Subsequently, BEEA [22] is proposed to overcome this shortcoming, using more iteration rounds to replace multiplication

ALGORITHM 1: Fermat's Little Theorem**Require:** Integer a , Modulus p **Ensure:** $a^{-1} \bmod p$

```

1:  $ret, k = 1, p - 2$ 
2: while  $k$  do
3:   if  $k$  is odd then
4:      $ret = ret \cdot a \bmod p$ 
5:   end if
6:    $k = k \gg 1$ 
7:    $a = a^2 \bmod p$ 
8: end while
9: return  $ret$ 

```

ALGORITHM 2: Extended Euclidean Algorithm**Require:** Integer a , Modulus p **Ensure:** $a^{-1} \bmod p$

```

1:  $x_1, x_2, r_1, r_2 = 0, 1, p, a$ 
2: while  $r_2 \neq 0$  do
3:    $q = \left\lfloor \frac{r_1}{r_2} \right\rfloor$ 
4:    $r_1, r_2 = r_2, r_1 - q \cdot r_2$ 
5:    $x_1, x_2 = x_2, x_1 - q \cdot x_2$ 
6: end while
7: return  $x_1 \bmod p$ 

```

and division with shift, addition and subtraction. Compared to EEA, BEEA is more suitable for hardware implementation.

MI uses the iterative structure and Montgomery multiplication/reduction to improve latency. For each integer a , it firstly uses the iteration to find an integer k and outputs $a^{-1} \cdot 2^k \bmod p$, and then uses Montgomery multiplication/reduction to get the final result $a^{-1} \bmod p$. In practical application, the integer should be a number in the Montgomery domain with the form of $\tilde{a} = a \cdot 2^l \bmod p$, where l is an integer and $2^l > p$. When, we use the MI to calculate $\tilde{a}$, the output should be $a^{-1} \cdot R \bmod p$, where $R = 2^l$. Algorithm 3 shows the detailed algorithm.

These algorithms have different advantages in different scenarios. Regarding computational performance, MI is typically more efficient than BEEA while FLT performs poorly. If we consider the hardware resource consumption, BEEA is superior to MI and FLT costs the most. However, FLT is still considered for the protection of private data because the number of iterations required for computation is independent of the input. This feature guarantees that FLT finishes the inversion in a constant time, while BEEA and MI do not.

3 Related Work

Yan [36] and Choi [6] used the form of radix-4 to improve the original BEEA and reduce the number of iteration rounds. Hossain [11], Zhou [37], and Mrabet [25] mainly focused on optimizing hardware designs. They shared the adders and reduced the delay caused by long carry chains. The implementation results showed that they not only improved the maximum frequency of the circuit but also reduced the computation latency. Bigou [2] first designed a modular inversion based on the **Residue Number System (RNS)**, which can decompose a large integer into multiple small

ALGORITHM 3: Montgomery Inversion

Require: Integer a , Modulus p
Ensure: $a^{-1} \bmod p$

```

1:  $k, u, v, r, s = 0, p, a, 0, 1$ 
2: while  $v > 0$  do
3:   if  $u$  is even then
4:      $u, s = u \gg 1, s \ll 1$ 
5:   else if  $v$  is even then
6:      $v, r = v \gg 1, r \ll 1$ 
7:   else if  $u > v$  then
8:      $u, r, s = (u - v) \gg 1, r + s, s \ll 1$ 
9:   else
10:     $v, s, r = (v - u) \gg 1, r + s, r \ll 1$ 
11:   end if
12:    $k = k + 1$ 
13: end while
14:  $r = p - r \bmod p$ 
15:  $b = \text{MontMul}(r, 2^{2^l - k}, p)$ 
16:  $ret = \text{MontRedc}(b, 2^l, p)$ 
17: return  $ret$ 

```

integers in different prime number domains. The addition and multiplication between small integers in RNS are independent, which allows for the design of parallel computation. Later in [3], Bigou performed further optimization by removing the comparison and used the plus-minus trick to reduce the number of modular operations. Hars [10] also used the plus-minus trick to improve Left-Shift [21], Right-Shift, and Shift Euclidean algorithms and analyzed their performance with the GMP library [18].

Deshpande [8] presented a sequential design and a full-length design, respectively. He also pointed out the limitation of full-length on hardware-constrained devices. When the bit length is less than 1,280, the full-length design performs better than the sequential one. As the bit length increases, the sequential design is more friendly to hardware. It is worth noting that the algorithm used in Deshpande's designs came from Bernstein [1], a constant time algorithm with adequate mathematical analysis. Besides, Murat [26] divided the full length into several blocks. Each smallest block accepts an 8-bit long input.

Kaliski [14] firstly proposed "almost Montgomery Inversion". Kaliski divided the computation into two phases. The first phase is similar to BEEA and requires iterations to get the specified result. The second phase is still iterative, but is called bit-level operation. Bos [4] proposed a method to keep the sum of the iteration rounds in the first and second phases constant. Savas [30] used Montgomery multiplication to replace the bit-level operation. The second phase is simplified by using this trick. In addition, the work of Savas was not limited to the integer or Montgomery domain. He also proposed cross-domain algorithms, but to the best of our knowledge, the algorithms have not yet been implemented on hardware.

FLT is the earliest constant-time modular inversion algorithm, and the calculation process is similar to **Modular Exponentiation Algorithm (MEA)**. The structure used for RSA encryption can also be applied to compute FLT. In the elliptic curve digital signature system designed by Lim [19], used the RNS-based FLT algorithm to solve the modular inversion for the generation of a private key. Xu [35] designed the $C(\cdot)$ function to re-express the prime numbers and used this

representation to calculate the least modular multiplication and exponentiation steps required by FLT. The new method could reduce the number of modular multiplication operations by about 90% without changing the number of powers, which improved the efficiency of using FLT to solve modular inversion.

In a word, there are various kinds of methods to optimize the calculation of modular inversion, e.g., high radix and RNS base. As SCAs continue to advance, it becomes necessary to ensure the security in the modular inversion calculation. Due to the low efficiency of FLT, more and more efficient and constant-time modular inversion algorithms have been proposed in recent years. In terms of algorithm effectiveness, most of these algorithms incorporate specific mathematical analyses to obtain complexity. For inputs with the same bit width, an algorithm with lower complexity implies higher effectiveness. However, the situation varies when considering the constant time feature. Most modular inversion algorithms involve multiple iterations and conditional branches. Different inputs may lead to varying numbers of iterations and branches, resulting in different execution time. The work by Joppe [4] introduced a reasonable criterion. It includes two aspects:

- The total computation time for a single iteration remains the same. This means that the longest computation time for a single iteration would involve calculating all branches and selecting the correct value, regardless of the number of branches within the iteration.
- The number of iterations is always the same when given a specific bit width. This means that the worst-case number of iterations is always required.

This article develops from the existing theories of previous work, especially [13, 30]. We proposed novel algorithms in constant time to resist the timing attack. We are also the first work to implement cross-domain algorithms on hardware. Our work not only extends the previous research, but also provides new directions and opportunities for future research.

4 Constant Time Modular Inversion

The reason why BEEA performs better than EEA on hardware is that BEEA makes a more appropriate tradeoff between the number of iteration rounds and the efficiency of each round. In this section, we will introduce our novel modular inversion algorithms, which take great advantage of some properties applied in BEEA and MI.

Our algorithms have two phases, which are similar to MI. In the first phase, the output form of our algorithms is different from MI. The result in MI is $a \cdot 2^k \bmod p$, while in our new algorithms, the result is $a \cdot 2^{-l} \bmod p$. a is the input but k, l are integers with different meanings. Besides, in order to resist the timing attack, the iteration rounds remain constant for the specified $GF(p)$ field. We named the first phase **Complementary Montgomery Iteration (CMI)**. As for the second phase, it has two types of configurations, which determine whether the whole algorithm is classic or cross-domain. We call the classic algorithm CT-IMI while the cross-domain algorithm CT-CMMI. CT-IMI and CT-CMMI have the same iteration structure. The differences between them are the form of the input number and the process in the second phase.

4.1 CMI Structure

In the following description, a_{init} and p_{init} correspond to the initial input number and the modulus. a and p are intermediate variables in the iterative process. In the beginning, $a = a_{init}$, $p = p_{init}$.

As we know, EEA is based on Bézout's identity. This lemma states that for integers a and p , there exist integers x, y such that

$$a \cdot x + p \cdot y = \gcd(a, p). \quad (4)$$

When p_{init} is a prime number, for each $a_{init} \in [1, p_{init})$, $\gcd(a_{init}, p_{init}) = 1$. According to Equation (4), there exist integers x, y so that $a_{init} \cdot x + p_{init} \cdot y = 1$, which is equivalent to $a_{init} \cdot x \equiv 1 \pmod{p_{init}}$. Then the value of x , also called the modular inversion of a_{init} , can be solved with an iterative method.

As we mentioned before, BEEA performs better than EEA since it replaces division and multiplication with addition, subtraction and shift operations. This optimization mainly depends on the following properties in number theory. The proof of these properties can be referred to [17].

Assume the integers a, p ($a < p$) are co-prime. When both a and p are odd:

$$\gcd(a, p) = \gcd\left(a, \frac{p-a}{2}\right). \quad (5)$$

When a is odd and p is even:

$$\gcd(a, p) = \gcd\left(a, \frac{p}{2}\right). \quad (6)$$

When a is even and p is odd:

$$\gcd(a, p) = \gcd\left(\frac{a}{2}, p\right). \quad (7)$$

For all a and p :

$$\gcd(a, p) = \gcd(a, p-a). \quad (8)$$

Different from BEEA, we introduce a new method for calculating x based on these properties. We choose one case for detailed derivation and other situations are similar to this.

Assume that the input $a_0 = a_{init}$ is even and $p_0 = p_{init}$ is odd since the modulus is a prime. Then, we combine Bézout's identity Equation (4) with the property Equation (7). There exist integers x_0, y_0, x_1 and y_1 which satisfy the following equation:

$$a_0 \cdot x_0 + p_0 \cdot y_0 = \gcd\left(\frac{a_0}{2}, p_0\right) = \frac{a_0}{2} \cdot x_1 + p_0 \cdot y_1 = a_0 \cdot \frac{x_1}{2} + p_0 \cdot y_1. \quad (9)$$

According to (9), $x_0 = \frac{1}{2}x_1, y_0 = y_1$ is one of the solutions. The relationship between x_0, y_0, x_1 and y_1 can be expressed more simply in the form of matrices Equation (10).

$$\begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \pmod{p_{init}}. \quad (10)$$

Similarly, now we can treat $a_1 = \frac{a_0}{2}$ and $p_1 = p_0$ as input. The transform Equation (11) is similar to Equation (10).

$$\begin{bmatrix} a_1 & p_1 \end{bmatrix} = \begin{bmatrix} a_0 & p_0 \end{bmatrix} \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{bmatrix}. \quad (11)$$

If we ignore x_0, y_0 for now, the problem becomes calculating the inversion of a_1 and the result is x_1 . The parity of a_1 and p_1 will suit one of the properties in Equations (5)–(7) again. Then an iterative process appears. In the i th iteration round, we denote the input as a_i, p_i , and there must exist x_i, y_i that satisfy the equation $a_i \cdot x_i + p_i \cdot y_i = 1$. If x_i, y_i can be fixed, we can obtain the final result x_0 through reverse iterative calculation. Besides, we stipulate that $a_i < p_i$ during the iteration, and if $a_i > p_i$ during the iteration, we exchange their values, as explained later.

Based on the properties from Equations (5) to (7), the parity of a_i, p_i will suit one of the situations in the i th iteration round, and it reduces at least one bit of a_i or p_i . After at most $n = \lceil \log_2 a + \log_2 p \rceil$ iteration rounds, the values of a_n and p_n will become 0 and 1. If we do not stipulate $a_i < p_i$, it is impossible to determine which of a_n and p_n becomes zero, and the situation will become more complex. Once we set that a_n and p_n become 0 and 1 finally, the value of x_n can be any and y_n must be 1 since the equation $a_n \cdot x_n + p_n \cdot y_n = 1$ still holds. The value of y_n is unique and definite in this case.

We assume that the values of a_k and p_k become 0 and 1 after k ($k \leq n$) rounds. Their values will not change anymore since the parity satisfies properties Equation (7). It means that the redundant

iteration rounds from $k + 1$ to n will not affect the correctness of the result. For different input a , the iteration round k may be different, while it can continue to iterate until the n rounds. Therefore, the iteration round is constant now. The constant round also means constant time.

Now we have solved the constant time problem, but the execution efficiency of the algorithm is still considerable. The basic idea is to reduce the upper boundary n when a_n, p_n can become 0 and 1. We observe that only a_i or p_i changes when using Equations (5), (6), and (7) properties in i th iteration round. Intuitively, if both of them decrease, or one of them decreases more than before, the upper boundary will also decrease. Since property Equation (8) is suitable for any a_i, p_i , we combine it with Equations (5), (6), and (7), and get the Equations (13), (14), and (15). Experimental results prove that this method is indeed effective and we will provide more details to determine the upper boundary in Section 6. The definition of the function $\mathcal{F}(\cdot)$ is as follows, which is a simplified form used in Equations (13), (14), and (15).

$$\mathcal{F}(m, n) = \begin{cases} m & m < n \\ m - n & m \geq n. \end{cases} \quad (12)$$

When both a and p are odd:

$$\gcd(a, p) = \gcd\left(a, \mathcal{F}\left(\frac{p-a}{2}, a\right)\right). \quad (13)$$

When a is odd and p is even:

$$\gcd(a, p) = \gcd\left(a, \mathcal{F}\left(\frac{p}{2}, a\right)\right). \quad (14)$$

When a is even and p is odd:

$$\gcd(a, p) = \gcd\left(\frac{a}{2}, \mathcal{F}\left(p - a, \frac{a}{2}\right), \frac{a}{2}\right). \quad (15)$$

Based on the Equations (13), (14), and (15) properties and Bézout's identity, we define the i th iteration round for a_i, p_i :

$$\begin{bmatrix} a_{i+1} & p_{i+1} \end{bmatrix} = \begin{bmatrix} a_i & p_i \end{bmatrix} \begin{bmatrix} \alpha_i & \beta_i \\ \gamma_i & \omega_i \end{bmatrix}. \quad (16)$$

Where $a_0 = a_{init}, p_0 = p_{init}$.

Multiplying $\begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix}$ on both sides of Equation (16), then

$$1 = \begin{bmatrix} a_{i+1} & p_{i+1} \end{bmatrix} \cdot \begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} \bmod p_{init} = \begin{bmatrix} a_i & p_i \end{bmatrix} \begin{bmatrix} \alpha_i & \beta_i \\ \gamma_i & \omega_i \end{bmatrix} \cdot \begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} \bmod p_{init}. \quad (17)$$

Since $a_i \cdot x_i + p_i \cdot y_i = a_{i+1} \cdot x_{i+1} + p_{i+1} \cdot y_{i+1} = 1$, then

$$\begin{bmatrix} x_i \\ y_i \end{bmatrix} = \begin{bmatrix} \alpha_i & \beta_i \\ \gamma_i & \omega_i \end{bmatrix} \begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} \bmod p_{init}. \quad (18)$$

We define the matrix $\begin{bmatrix} \alpha_i & \beta_i \\ \gamma_i & \omega_i \end{bmatrix}$ as the **Transition Matrix (TM)** for the i th iteration round. The i th TM is decided by the parity and values of a_i, p_i , as shown in Table 1. Property (13) corresponds to three cases in Table 1 since we not only compare the value of $\frac{p-a}{2}$ and a , but also a and $\mathcal{F}(\frac{p-a}{2}, a)$ to promise $a_i < p_i$ in each iteration round. Property Equations (14) and (15) are similar.

Table 1. Relationship between a_i , p_i and TM

Parity of a_i, p_i	Value of a_i, p_i	TM	α_i	β_i	γ_i	ω_i
a_i :odd, p_i :odd	$p_i \geq 5a_i$	M_0	1	$-\frac{3}{2}$	0	$\frac{1}{2}$
	$3a_i \leq p_i < 5a_i$	M_1	$-\frac{3}{2}$	1	$\frac{1}{2}$	0
	$a_i < p_i < 3a_i$	M_2	$-\frac{1}{2}$	1	$\frac{1}{2}$	0
a_i :odd, p_i :even	$p_i \geq 4a_i$	M_3	1	-1	0	$\frac{1}{2}$
	$2a_i \leq p_i < 4a_i$	M_4	-1	1	$\frac{1}{2}$	0
	$a_i < p_i < 2a_i$	M_5	0	1	$\frac{1}{2}$	0
a_i :even, p_i :odd	$p_i \geq \frac{5}{2}a_i$	M_6	$\frac{1}{2}$	-2	0	1
	$2a_i \leq p_i < \frac{5}{2}a_i$	M_7	-2	$\frac{1}{2}$	1	0
	$\frac{3}{2}a_i \leq p_i < 2a_i$	M_8	$\frac{1}{2}$	-1	0	1
	$a_i < p_i < \frac{3}{2}a_i$	M_9	-1	$\frac{1}{2}$	1	0

Based on the iteration formula Equation (18), we can expand the whole chains to calculate x_0 as follows:

$$\begin{aligned}
 \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} &= \begin{bmatrix} \alpha_0 & \beta_0 \\ \gamma_0 & \omega_0 \end{bmatrix} \dots \begin{bmatrix} \alpha_i & \beta_i \\ \gamma_i & \omega_i \end{bmatrix} \dots \begin{bmatrix} \alpha_{n-1} & \beta_{n-1} \\ \gamma_{n-1} & \omega_{n-1} \end{bmatrix} \cdot \begin{bmatrix} x_n \\ y_n \end{bmatrix} \bmod p_{init} \\
 &= \frac{1}{2^n} \begin{bmatrix} 2\alpha_0 & 2\beta_0 \\ 2\gamma_0 & 2\omega_0 \end{bmatrix} \dots \begin{bmatrix} 2\alpha_i & 2\beta_i \\ 2\gamma_i & 2\omega_i \end{bmatrix} \dots \begin{bmatrix} 2\alpha_{n-1} & 2\beta_{n-1} \\ 2\gamma_{n-1} & 2\omega_{n-1} \end{bmatrix} \cdot \begin{bmatrix} x_n \\ y_n \end{bmatrix} \bmod p_{init}. \quad (19)
 \end{aligned}$$

n is the iteration round. For each TM, we convert the coefficients to integers by doubling their values. This process avoids the modular shift operation, which means that the odd number needs to add p_{init} before shifting. We mentioned that after n iteration rounds, the values of a_n and p_n become 0 and 1, respectively. And the value of y_n is always 1, while the value of x_n can be any. In order to simplify the calculation of x_0 , we set $x_n = 0$.

According to Equation (19), we get the following expression to calculate x_0 :

$$\begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \frac{1}{2^n} \begin{bmatrix} \alpha & \beta \\ \gamma & \omega \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \bmod p_{init}, \quad (20)$$

where $\begin{bmatrix} \alpha & \beta \\ \gamma & \omega \end{bmatrix} = \begin{bmatrix} 2\alpha_0 & 2\beta_0 \\ 2\gamma_0 & 2\omega_0 \end{bmatrix} \dots \begin{bmatrix} 2\alpha_{n-1} & 2\beta_{n-1} \\ 2\gamma_{n-1} & 2\omega_{n-1} \end{bmatrix}$.

Then $x_0 = \frac{\beta}{2^n} \bmod p_{init}$, where β is the number calculated during the iterative process. The above iteration process is the first phase of our new algorithms, and the second phase further processes β to get the final modular inversion result x_0 .

Algorithm 4 shows the details. In each iteration round, we need to choose the TM firstly according to the parity and values of a_i, p_i , which corresponds to the operation in line 4. Then we can use the Equations (18) and (16) iteration formulas, which are shown in lines 5 and 6. We require an

initial matrix before getting the first TM, and an identity matrix $E = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ is satisfied. After we get the first TM $M_0 = \begin{bmatrix} \alpha_0 & \beta_0 \\ \gamma_0 & \omega_0 \end{bmatrix}$, we can multiply E by M_0 . For the next round, we get M_1 and keep multiplying. Finally, we get the value of $\begin{bmatrix} \alpha & \beta \\ \gamma & \omega \end{bmatrix}$. Since only the value of β is useful, we can use a determinant $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}_{1 \times 2}$ to replace matrix E . In Algorithm 4, a, p and u, v are intermediate variables, and we express these variables in determinants for simplicity. The meaning of lines 5 and 6 is to multiply a determinant and a matrix, updating the result in determinants A, B .

ALGORITHM 4: Complementary Montgomery Iteration (CMI)

Require: Integer a_{init} , Modulus p_{init} , Iteration round n

Ensure: v

- 1: $a = a_{init}, p = p_{init}, u = 1, v = 0, k = 0$
 - 2: $A = \begin{bmatrix} u & v \end{bmatrix}_{1 \times 2}, B = \begin{bmatrix} a & p \end{bmatrix}_{1 \times 2}$
 - 3: **while** $k \neq n$ **do**
 - 4: choose $M_t, t \in [0, 9]$ according to Table 1
 - 5: $A = A \cdot 2M_t$
 - 6: $B = B \cdot M_t$
 - 7: **end while**
 - 8: **return** v
-

4.2 CT-IMI and CT-CMMI

In Section 4.1, we have shown the iteration formula, by multiplying TM n times to get β . But the result β is not the final inversion x_0 , which requires further processing.

Since β cannot directly remove the lowest n bits, the simplest way is using the modular shift operation. This operation adds the odd number with p_{init} before shifting or directly shifts when the number is even. Algorithm 5 describes this process. Using this method also requires n iteration rounds. It is not efficient even though it has a constant time feature.

ALGORITHM 5: Bit-Level Operation Reduction

Require: Integer $v = a \cdot 2^{-n}$, Modulus p_{init}

Ensure: $a^{-1} \bmod p_{init}$

- 1: $k = 0, ret = v$
 - 2: **while** $k \neq n$ **do**
 - 3: **if** v is even **then**
 - 4: $ret = ret \gg 1$
 - 5: **else**
 - 6: $ret = (ret + p_{init}) \gg 1$
 - 7: **end if**
 - 8: $k = k + 1$
 - 9: **end while**
 - 10: **return** ret
-

Fortunately, **MontRedc**($\cdot$) can simplify this process, since $\mathbf{MontRedc}(\beta) = \frac{\beta}{2^n}$. If we combine CMI and **MontRedc**($\cdot$), it may be helpful to compute the inversion in the integer field. Assume that for integer a'_{init} and its Montgomery representation a_{init} , $a_{init} = a'_{init} \cdot 2^l \bmod p_{init}$. If we use a'_{init} as

the input number, we can calculate β' after the CMI process. If the input number is a_{init} now, we get β . The following equations are held.

$$a_{init} \cdot \frac{\beta}{2^n} + p_{init} \cdot \frac{\omega}{2^n} = 1, \quad (21)$$

$$a'_{init} \cdot \frac{\beta'}{2^n} + p_{init} \cdot \frac{\omega'}{2^n} = 1. \quad (22)$$

Then $\beta \bmod p_{init} = 2^l \cdot \beta' \bmod p_{init}$. When l is equal to n , we directly get $x_0 = \beta' \bmod p_{init}$, which is the inversion of a_{init} . The value of l only needs to be satisfied with $2^l > p_{init}$, and the value of n based on CMI is greater than $\log_2 p_{init}$, which we can set $l = n$. Now, we can use a Montgomery domain number as the input of CMI, and get the inversion in the integer field. Modular reduction is used here to limit the output in a finite field.

ALGORITHM 6: Montgomery Reduction

Require: Integer v , Modulus p_{init} , Iteration round n , Integer $r := 2^n$, and $p_{inv} := -p_{init}^{-1} \bmod r$.

Ensure: $v \cdot r^{-1} \bmod p_{init}$

- 1: $m = ((v \bmod r) \cdot p_{inv}) \bmod r$
 - 2: $ret = (v + m \cdot p_{init}) \gg n$
 - 3: **return** ret
-

Algorithm 6 shows the details of Montgomery Reduction. p_{inv} is pre-computed and the multiplication steps can be simplified. The operation $\bmod r$ can also be replaced by bit-and operation due to r is 2^n .

ALGORITHM 7: Modular Reduction for P_{sm2}

Require: Integer v

Ensure: $v \bmod P_{sm2}$

- 1: $(v_9, v_8, v_7, v_6, v_5, v_4, v_3, v_2, v_1, v_0)_{10 \times 32} = v$
 - 2: $r = (v_9 + v_8 + v_7, v_6, v_5, v_9 + v_4, v_8 + v_3, v_2 - v_8 - v_9, v_1 + v_9, v_0 + v_9 + v_8)$
 - 3: $(r_8, r_7, r_6, r_5, r_4, r_3, r_2, r_1, r_0)_{9 \times 32} = r$
 - 4: $ret = (r_7 + r_8, r_6, r_5, r_4, r_3 + r_8, r_2 - r_8, r_1, r_0 + r_8)$
 - 5: **return** ret
-

Algorithm 7 shows an optimization method for modular reduction, which uses the congruence properties of some specific primes. Here we show the case for the prime used in SM2. The following congruence properties are used.

$$2^{256} \equiv 2^{224} + 2^{96} - 2^{64} + 1 \bmod P_{sm2}$$

$$2^{288} \equiv 2^{224} + 2^{128} - 2^{64} + 2^{32} + 1 \bmod P_{sm2}$$

The prime in SM2 can be expressed as the sum or difference of a number of powers of 2:

$$p_{sm2} = 2^{256} - 2^{224} - 2^{96} + 2^{64} - 1$$

Let v be a 320-bit integer (the statistical results of experiments for P_{sm2} showed the value of v is no more than 320 bits) and the base-2³² representation of v can be denoted as $v = \sum_{i=0}^9 v_i \cdot 2^{32 \cdot i}$, where $v_i \in [0, 2^{32} - 1]$.

Then the higher powers of 2 in v can be reduced using the above congruence properties:

$$\begin{aligned}
 r &= v \bmod P_{sm2} = v_9 \cdot 2^{288} + \dots + v_1 \cdot 2^{32} + v_0 \bmod P_{sm2} \\
 &= v_9 \cdot (2^{224} + 2^{128} - 2^{64} + 2^{32} + 1) \\
 &\quad + v_8 \cdot (2^{224} + 2^{96} - 2^{64} + 1) + v_7 \cdot 2^{224} + \dots + v_0 \bmod P_{sm2} \\
 &= (v_9 + v_8 + v_7) \cdot 2^{224} + \dots + (v_1 + v_9) \cdot 2^{32} + (v_0 + v_9 + v_8) \bmod P_{sm2}
 \end{aligned}$$

Since the value of r is still larger than P_{sm2} , we can continue this process. We write the base- 2^{32} representation of r and reduce the higher powers of 2 in r :

$$\begin{aligned}
 ret &= r \bmod P_{sm2} = r_8 \cdot 2^{256} + \dots + r_1 \cdot 2^{32} + r_0 \bmod P_{sm2} \\
 &= r_8 \cdot (2^{224} + 2^{96} - 2^{64} + 1) + r_7 \cdot 2^{224} + \dots + r_0 \bmod P_{sm2} \\
 &= (r_8 + r_7) \cdot 2^{224} + \dots + r_1 \cdot 2^{32} + (r_0 + r_8) \bmod P_{sm2}
 \end{aligned}$$

For ret , it only requires at most one-time extra subtraction to get the final reduction result. In summary, we give a full description of our modular inversion algorithms in Algorithm 8.

ALGORITHM 8: CT-IMI and CT-CMMI

Require: Integer $a_{init} = a'_{init} \cdot 2^n$, Modulus p_{init} , Iteration round n

Ensure: a_{init}^{-1} or $a'_{init}{}^{-1} \bmod p_{init}$

- 1: $v = \text{CMI}(a_{init}, p_{init})$
 - 2: **if** $MODE = \text{CT-IMI}$ **then**
 - 3: $ret = \text{MontRedc}(v)$
 - 4: **else if** $MODE == \text{CT-CMMI}$ **then**
 - 5: $ret = \text{Redc}(v)$
 - 6: **end if**
 - 7: **return** ret
-

5 Hardware Architecture

In this section, we propose hardware architectures to implement our algorithms, including a universal iteration architecture, Montgomery and modular reduction for P_{sm2} .

Based on the algorithms of CT-IMI and CT-CMMI, we design two hardware architectures on Xilinx Virtex-7 (xc7vx690tffg1157-3) and Zynq-Ultrascale+ (xczu19eg-ffvb1517-3-e) FPGA. Our designs are synthesized using Xilinx Vivado 2018.3 synthesis tools with the “Area Optimized high” directive. Figure 1 shows the top architecture. Since the iteration architecture is the same, the difference between the two designs is the optional architecture, which is marked in Figure 1. In general, the modular reduction can be calculated in three ways:

- General modulus forms (i.e., Barrett and Montgomery algorithms), which are slower but suitable for more moduli.
- Based on the LUT method (which is based on pre-computed values) but requires a large amount of memory.
- Modulo form of prime numbers, such as pseudo-Mersenne numbers. The special form makes them appropriate for modular reduction.

In our design, we choose the Montgomery reduction for extensibility and the modular reduction with specific optimization for some primes for high efficiency.

In this work, we use the prime P_{sm2} to finish the whole modular inversion architecture. Four registers are mainly used during the iteration. At the beginning of the iteration, 256 bits registers

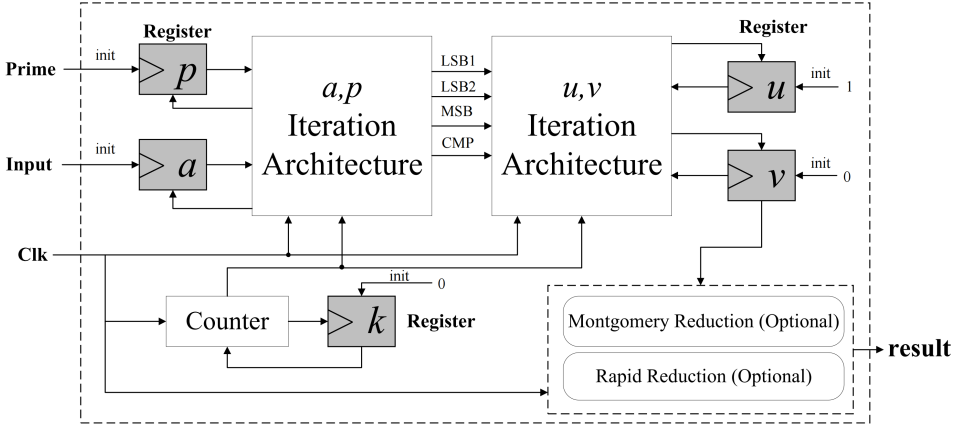


Fig. 1. The top hardware architecture of modular inversion.

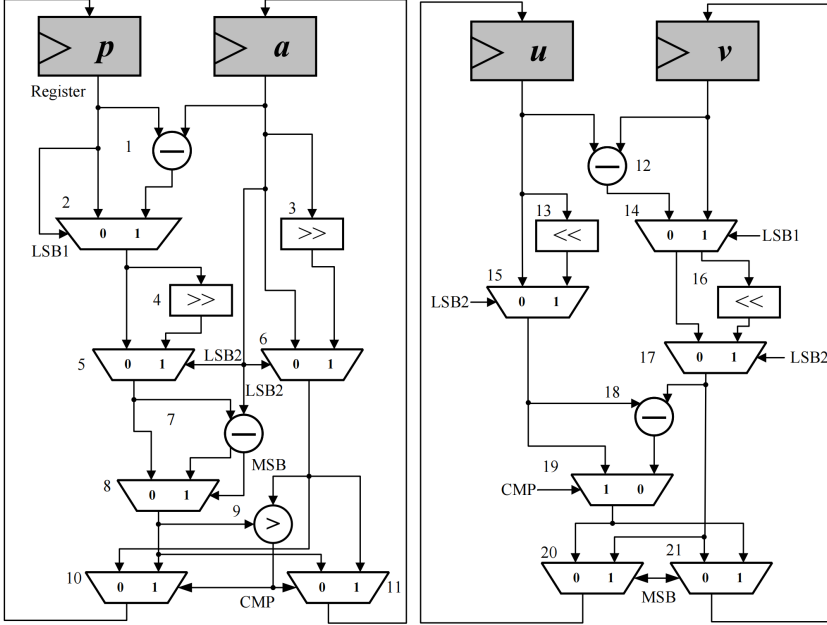


Fig. 2. The CMI hardware architecture.

a and p are used to store the input number and P_{sm2} . 320 bits registers u and v are initialized to 1 and 0. Another 9-bit register k is used as a counter. The matrix multiplication is simplified to shifting and subtraction operations. LSB1 and LSB2 are the **Least Significant Bit (LSB)** which reflect the parties. MSB is the Most Significant Bit that reflects whether the result is positive or not. This bit helps to exchange the value of a_i and p_i when $a_i > p_i$. CMP is the result of a comparer that equals to 1 or 0. They are used in multiplexers. In each iteration round, registers a , p , u , v will update the value and the update logic designed in Figure 2 is based on Table 1. The counter also increases by 1 in register k at the same time. Each iteration is completed in one clock cycle. After 281 times iteration, the iteration will stop and the value in register v should remain unchanged. In

Table 2. The Corresponding Iteration Relationship of Registers a, p, u, v

a is odd	p is odd	Value Relationship	a, p update	u, v update
✓	✓	$p > 5a$	$a = a, p = \frac{p-3a}{2}$	$u = 2u, v = v - 3u$
✓	✓	$3a < p < 5a$	$a = \frac{p-3a}{2}, p = a$	$u = v - 3u, v = 2u$
✓	×	$p > 4a$	$a = a, p = \frac{p-2a}{2}$	$u = 2u, v = v - 2u$
✓	×	$2a < p < 4a$	$a = \frac{p-2a}{2}, p = a$	$u = v - 2u, v = 2u$
✓	✓	$a < p < 3a$	$a = \frac{p-a}{2}, p = a$	$u = v - u, v = 2u$
✓	×	$a < p < 2a$	$a = \frac{p}{2}, p = a$	$u = v, v = 2u$
×	✓	$p > \frac{5}{2}a$	$a = \frac{a}{2}, p = a$	$u = u, v = 2v - 4u$
×	✓	$2a < p < \frac{5}{2}a$	$a = p - 2a, p = \frac{a}{2}$	$u = 2v - 4u, v = u$
×	✓	$\frac{3}{2}a < p < \frac{5}{2}a$	$a = \frac{a}{2}, p = p - a$	$u = u, v = 2v - 2u$
×	✓	$a < p < \frac{3}{2}a$	$a = p - a, p = \frac{a}{2}$	$u = 2v - 2u, v = u$

each iteration, the values in registers a, p, u, v are updated and the corresponding results are shown in Table 2.

In Figure 2, We have labeled each basic component. Detailed descriptions are as follows. Labels 1, 7, 12, 18 mark subtractors. Labels 2, 5, 6, 8, 10, 11, 15, 17, 20, 21 represent **multiplexers (MUX)** and labels 3, 4, 13, 16 mark shifters. Label 9 marks the comparer. Since LSB1 and LSB2 are the least significant bits of a, p respectively, they can be used to judge the parity of values in a, p . Registers a and p are directly connected to subtractor 1, and the output of subtractor 1 is the value of $p - a$. Thus, the output of MUX 2 is the value of p or $p - a$, which depends on LSB1. Then we make a right shift operation on the output of MUX 2 by shifter 4, and its result is one of the inputs of MUX 5. Considering several possible inputs are generated by MUX 2, the output of MUX 5 is one of the values in $p, p - a, \frac{p}{2}, \frac{p-a}{2}$, which depends on LSB2. The function $\mathcal{F}(\cdot)$ is implemented by subtractor 7 and MUX 5. We use subtractor 7 to calculate the difference between the output of MUX 5 and a . If the difference is negative, then the MSB will be 1. Otherwise, it will be 0. The MSB determines which of values in $p, p - a, p - 2a, \frac{p}{2}, \frac{p-a}{2}, \frac{p-2a}{2}, \frac{p-3a}{2}$ is the output of MUX 8. Finally, in order to ensure the value of p is larger than a , we use comparer 9 to make a comparison between the output of MUX 8 and MUX 6. The larger one will be stored in register p while the other will be stored in register a . The update process of p, a is finished by MUX 10 and 11, where the CMP is the control signal. The update process of u, v is similar to that of p, a , and we have sorted out all the possible outputs of each basic component in Table 3. When the iteration is finished, we will use Montgomery reduction to get the final result if the input is an integer according to the algorithm mentioned in Section 4.2. Otherwise, modular reduction is needed since the maximum value in register v may reach 320 bits.

Table 3. The Possible Outputs of Each Basic Component

Label	Possible Outputs	Label	Possible Outputs
1	$p - a$	12	$v - u$
2	$p, p - a$	13	$2u$
3	$\frac{a}{2}$	14	$v, v - u$
4	$\frac{p}{2}, \frac{p-a}{2}$	15	$u, 2u$
5	$p, p - a, \frac{p}{2}, \frac{p-a}{2}$	16	$2v, 2v - 2u$
6	$a, \frac{a}{2}$	17	$v, v - u, 2v, 2v - 2u$
7	$p - a, p - 2a, \frac{p-2a}{2}, \frac{p-3a}{2}$	18	$v - u, v - 2u, v - 3u, 2v - 2u, 2v - 4u$
8	$\frac{p}{2}, \frac{p-a}{2}, \frac{p-2a}{2}, \frac{p-3a}{2}, p - a, p - 2a$	19	$u, 2u, v - u, v - 2u, v - 3u, 2v - 2u, 2v - 4u$
10	$a, \frac{a}{2}, p - a, p - 2a,$	20	$u, 2u, v, v - u, v - 2u,$ $v - 3u, 2v - 2u, 2v - 4u$
11	$\frac{p}{2}, \frac{p-a}{2}, \frac{p-2a}{2}, \frac{p-3a}{2}$	21	

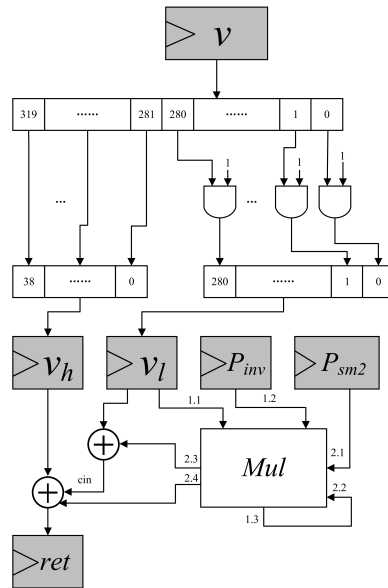


Fig. 3. The montgomery reduction for P_{sm2} hardware architecture.

Algorithm 6 and its corresponding hardware architecture in Figure 3 are designed to finish the Montgomery reduction. Let $r = 2^{281}$ and $v \bmod r$ be finished with the bit-and operation. We put the lower 281 bits of v into the v_l register, and the higher 39 bits of v into the v_h register. Since $P_{inv} = -P_{sm2}^{-1}$, it can be pre-computed, and the multiplication steps in Algorithm 6 can be simplified. For the sake of brevity, we only draw one “Mul” in Figure 3, and use the symbols to distinguish inputs and outputs. The first multiplication operation is $v_l(1.1) \cdot P_{inv}(1.2)$. The whole multiplication result (1.3) will reach 537-bit, which should be reduced in modulo r . Thus, only the lower 281-bit value is effective and this saves a lot of hardware resources. The second multiplication is the

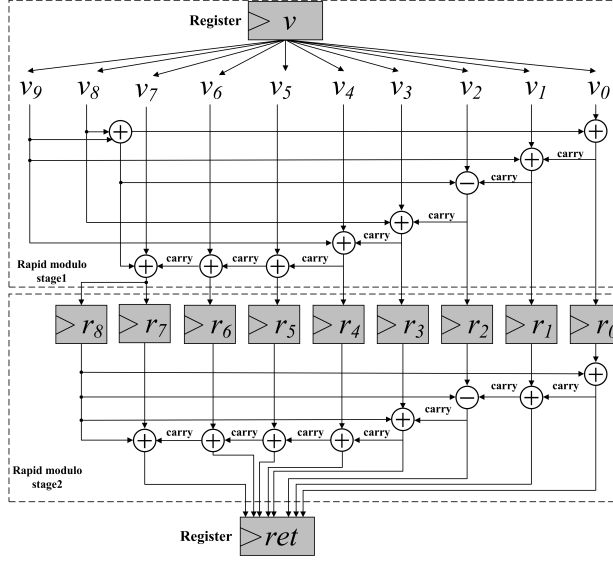


Fig. 4. Pipelined modular reduction for P_{sm2} hardware architecture.

result from (1.3) and $P_{sm2}(2.1)$. Since it only consists of a small number of powers of 2 ($2^{P_{sm2}} = 2^{256} - 2^{224} - 2^{96} + 2^{64} - 1$), we directly convert this multiplication process into shift-and-add operations. The output in (2.3) is the lower 281 bits, and (2.4) is the higher 256 bits. Then we add up the number in v_l and (2.3), and only take out the carry part (cin) of the operation result exceeding 281 bits, which can avoid the last shift operation in Algorithm 6. Finally, the sum of v_h , cin and (2.4) is the result of modular inversion. At most one subtraction is required to ensure that the result is constrained within 256 bits.

Algorithm 7 and its corresponding hardware architecture in Figure 4 are designed to finish the modular reduction. In order to increase the maximum frequency of the circuits, registers $r_0, r_1, \dots, r_8$ are used to divide the modular reduction into two stages and form a two-stage pipeline.

In the first stage, the value of 320-bit register v is divided into ten 32-bit numbers $v_0, v_1, \dots, v_9$, such that $v = v_9 \cdot 2^{288} + v_8 \cdot 2^{256} + \dots + v_0$. A carry chain is necessary for registers $r_0, r_1, \dots, r_8$ whose bit length is 32. For example, if $w_0 = v_0 + v_8 + v_9$, the bit length of w_0 will reach 33. Therefore, we divide w_0 into a one-bit carry and a 32-bit sum, such that the carry $c_0 = w_0 \gg 32$ and the sum $s_0 = w_0 \& 0xFFFFFFFF$. The sum s_0 is stored in register r_0 , and the carry is used to calculate $w_1 = v_1 + v_9 + c_0$. Ultimately, the ten numbers $v_0, \dots, v_9$ are transformed into nine numbers $r_0, r_1, \dots, r_8$. In fact, if we let $r = r_8 \cdot 2^{256} + \dots + r_0$, it follows that $r \bmod p = v \bmod p$, and the bit length of r is at most 288.

In the second stage, the bit length of the register ret is 256, which is similar to that of the first stage. We perform addition, subtraction, and carry processing with the numbers in registers $r_0, \dots, r_8$, and finally, obtain eight 32-bit numbers that can split the 256 bits into the result as the output of modular inversion.

Considering that the iteration part is the core of our module, we analyzed its critical paths. The critical paths are highlighted in Figures 5–7. We can define the following delays:

- D_a : the maximal delay from input a to a_{next} flip-flop, via two subtractors, four MUX, one shifter and one comparator;

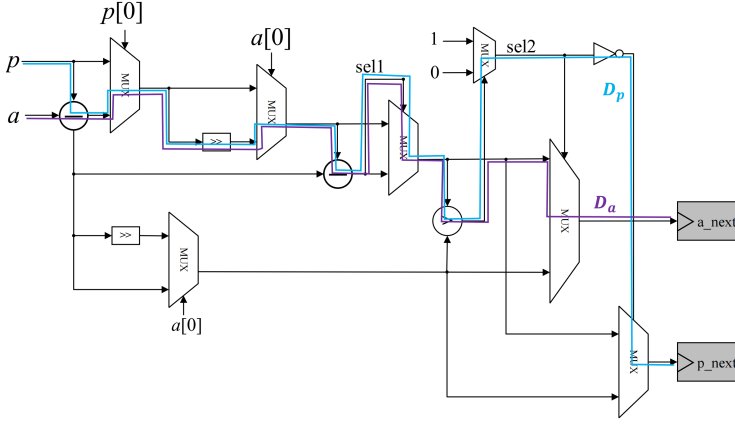


Fig. 5. Overview of the critical paths in a, p iteration architecture.

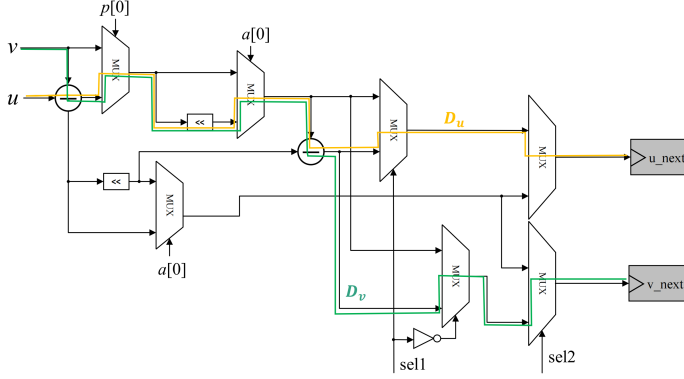


Fig. 6. Overview of the critical paths in u, v iteration architecture.

- D_p : the maximal delay from input p to p_next flip-flop, via two subtractors, five MUX, one shifter and one inverter;
- D_u : the maximal delay from input u to u_next flip-flop, via two subtractors, four MUX and one shifter;
- D_v : the maximal delay from input v to v_next flip-flop, via two subtractors, four MUX and one shifter;
- D_w : the maximal delay from input to next flip-flop, via two subtractors, one shifter, five MUX, one comparer and one inverter.

Note that we only consider the longest paths for each register a_next , p_next , u_next and v_next in Figures 5 and 6. The critical path for the whole iteration architecture is marked red in Figure 7.

6 Experimental Results and Comparisons

In this section, we will show the experiments on both software and hardware. We first analyze the least iteration rounds for different bit length numbers and make a comparison with the other algorithms. Since the performance of CT-IMI and CT-CMMI is similar in software, we only use CT-IMI for software comparison. Then the specific designs for P_{sm2} on FPGA are shown and we make a comparison with the other hardware implementations on 256-bit length numbers.

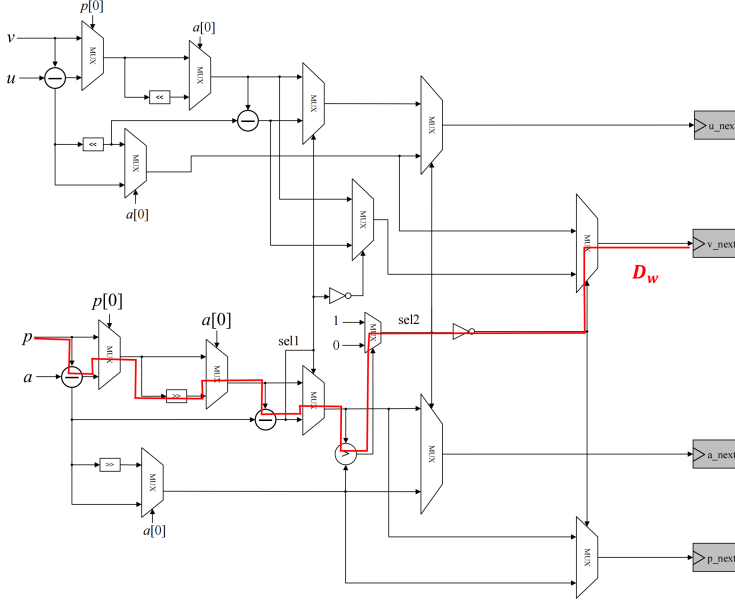


Fig. 7. Overview of the critical path in the whole iteration architecture.

6.1 The Constant Iteration Round

The question remaining from Section 4 is the value of iteration round n . Given that it is too redundant even though setting $n = \lceil \log_2 a + \log_2 p \rceil$, strict mathematical proofs of boundaries for generic primes are left for future work. Here we selected some commonly used prime numbers $P_{192}, P_{224}, P_{256}, P_{384}, P_{521}, P_{1279}, P_{2203}$ in ECC [5] published by NIST for experiments. We created 10^7 random inputs as a by the “secrets” library of Python, which are true random numbers generated by the operating system. For each prime, we count the iteration rounds for different inputs when the values of a_i and p_i become 0, 1. The maximum result will be the upper boundary of this prime. The values of these primes are shown as follows:

$$P_{192} = 2^{192} - 2^{64} - 1$$

$$P_{224} = 2^{224} - 2^{96} + 1$$

$$P_{256} = 2^{256} - 2^{224} - 2^{96} + 2^{64} - 1$$

$$P_{384} = 2^{384} - 2^{128} - 2^{96} + 2^{32} - 1$$

$$P_{521} = 2^{521} - 1$$

$$P_{1279} = 2^{1279} - 1$$

$$P_{2203} = 2^{2203} - 1$$

We provide an example to illustrate the simulation process. All codes and statistics are recorded in <https://github.com/Aimmecat/CTCMI>. The software experimental platform uses Python 3.7 on Intel (R) Core (TM) i7-9700K CPU @ 3.6GHz, 32.0 GB RAM and the operating system is Ubuntu 18.04.6 LTS (GNU/Linux 4.15.0-172-generic x86_64). When the prime is P_{192} , we create 10^7 random inputs $a_j, j \in [1, 10^7], a_j \in [1, 2^{192}]$. In Section 4.1, the inversion of each a_j can be expanded as a series of matrices, but the number of matrices is not determined. There exists a boundary and the redundant matrices will not affect the correctness of the result. Then we use our novel algorithms

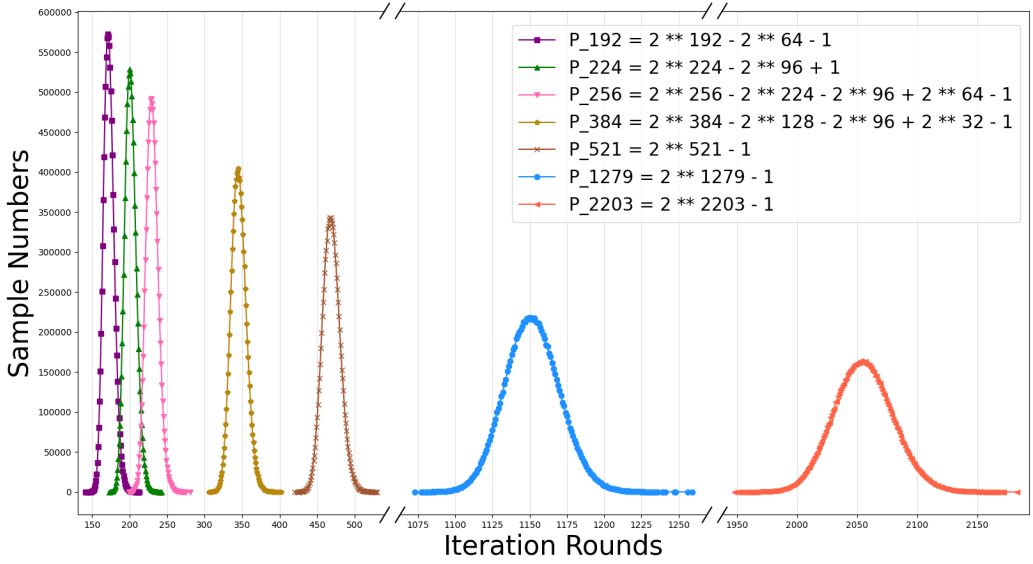


Fig. 8. The distribution of iteration rounds with different primes.

to find the least iteration rounds for each a_j . Figure 8 displays the distribution of iteration rounds for various primes. The horizontal axis represents the number of iteration rounds, while the vertical axis represents the number of corresponding sampled inputs. According to Figure 8, out of a total of 10^7 inputs, approximately 5.8×10^5 of them have a number of iteration rounds equal to 172, and the boundary for P_{192} is near 193. It also means for P_{192} , the inversion of the most of inputs can be represented by the product of 193 matrices. The situation is similar for the other primes. For each random input, the minimum iteration rounds have been counted and recorded in Figure 8.

When we focus on each prime, the iteration rounds distribution is similar to a Gaussian distribution. To verify this, we employ Anderson–Darling and Shapiro–Wilk tests, but the results were inconclusive, as we cannot test all inputs, even for P_{192} . However, when we sampled 10^2 from 10^7 results, all primes passed the test. As the sample size increased to 10^3 , only P_{1279} and P_{2203} passed the test. Although we cannot ensure that it strictly satisfies a Gaussian distribution, we can still use it as a model to simplify the analysis and predict the upper boundary by fitting the curve to a Gaussian distribution.

Table 4 shows the fitting results. The amplitude is too large and we normalize the original data before fitting the mean μ and variance σ . MSE is also calculated to show the accuracy. The Range column records the shortest and longest iteration rounds. After fitting the μ and σ for each prime, the value of $\mu \pm 3\sigma$ is calculated. Compared to the Range column, the longest iteration rounds are larger than $\mu + 3\sigma$. We denote the longest iteration rounds and Gaussian distribution as $H, G(\cdot)$. Since the maximum iteration rounds n must be in $[1, \log_2 a + \log_2 p]$, then the probability of $Pr(n \leq H) = \frac{\sum_{i=1}^H G(i)}{\sum_{i=1}^{\log_2(a \cdot p)} G(i)}$. We are not using CDF to calculate $Pr(n \leq H)$ here since the distribution of the iteration round is discrete. The experiment results indicated that setting the H as the constant iteration round is reasonable. Although there may exist numbers that require more iteration rounds, the probability is almost negligible for practical applications. Also in some real world applications where we need to guarantee the correctness of the output, we can check whether it satisfies the termination condition after a sufficient number of iterations. Here, we choose H rather than $\lceil \log_2 a + \log_2 p \rceil$, which can be viewed as a tradeoff between efficiency and accuracy.

Table 4. Statistics of Iteration Rounds and Gaussian Distribution Fitting for Different Primes

Prime	Source	μ	σ	MSE	Range([L,H])	$\mu \pm 3\sigma$	Probability
P_{192}	NIST [28]	171.8	6.9	0.028	[141, 214]	[151.1, 192.6]	$1 - 2^{-30.1}$
P_{224}	NIST [28]	200.6	7.5	0.025	[173, 243]	[178.1, 223.3]	$1 - 2^{-27.3}$
P_{256}	SM2 [27]	229.5	8.1	0.024	[199, 281]	[205.3, 253.8]	$1 - 2^{-33.9}$
P_{384}	NIST [28]	344.8	9.9	0.022	[306, 402]	[315.2, 374.4]	$1 - 2^{-28.6}$
P_{521}	Mersenne	468.3	11.6	0.017	[420, 530]	[433.4, 503.1]	$1 - 2^{-24.5}$
P_{1279}	Mersenne	1,151.2	18.3	0.012	[1073, 1259]	[1096.4, 1206.0]	$1 - 2^{-29.3}$
P_{2203}	Mersenne	2,053.8	24.4	0.011	[1948, 2183]	[1980.9, 2127.0]	$1 - 2^{-24.2}$

6.2 Software Simulations

In Section 6.1, we analyze how we determine the constant iteration rounds. We choose some non-constant time modular inversion algorithms to show the relationship between the bit length and latency. Then we use these algorithms to make a comparison with our algorithm to show the necessity of the constant time feature. Figure 9 shows the results. For each bit length, we sample 10^4 numbers and calculate the average latency for one modular operation. The prime is changed with the bit length of the number. When the bit length is less than 193, P_{192} is used. While the bit length increases to 193, we will change the prime to P_{224} . The other situations are similar and we denote it in Figure 9. For each prime, the small picture in the upper left of Figure 9 shows the difference between mean and original latency. In addition to BEEA and MI, HBEEA [12], Stein [15] and P_stein are used for comparison.

The experimental results prove that CT-IMI performs better in security than other non-constant algorithms since the whole line is stable and the difference between mean and original latency is nearly 0 for each prime. For the same prime, there is little difference in operation latency between longer and shorter bit length numbers. This feature makes it resist the timing attack. Even though CT-IMI is strictly constant time in the algorithm, the operating frequency during software simulations will inevitably change, resulting in slight differences in latency. It is worth noting that for each prime, e.g., P_{2203} , it does not require that the input number must exceed 1,279 bits. When using a 1-bit number, the latency is still nearly indistinguishable from a 2,203-bit number. To make Figure 9 clear, we omit these parts.

It should be emphasized that since our novel algorithms have constant iteration rounds, the latency in software is nearly indistinguishable. The result in Figure 9 itself does not mean that our algorithms are strong enough to resist SCAs. We mainly use Figure 9 to illustrate that the non-constant time modular inversion algorithms have higher security risks than the constant time modular inversion algorithms. The side-channel security of our novel algorithm is promised by the constant iteration rounds, while software simulation just makes a verification.

There are many other constant algorithms, e.g., BOS [4], BY [1], hdBY [34]. In the latest available literature, Jin [13] showed that the average clock cycles of SICT-MI are lower than these algorithms. In Table 5, we make a comparison with these constant modular inversion algorithms. Compared to the latest SICT-MI, CT-IMI saves around 30% clock cycles for each prime. This is mainly due to the choice of iteration rounds. SICT-MI used $\lceil 2 \log_2 p \rceil$ as the constant rounds, which is conservative, while we analyze the distribution of iteration rounds and choose a lower boundary for efficiency. Since we used Python for experiments, the clock cycles of SICT-MI are larger than the results in the original article. The growth rate of clock cycles consumed by FLT is much higher than those of

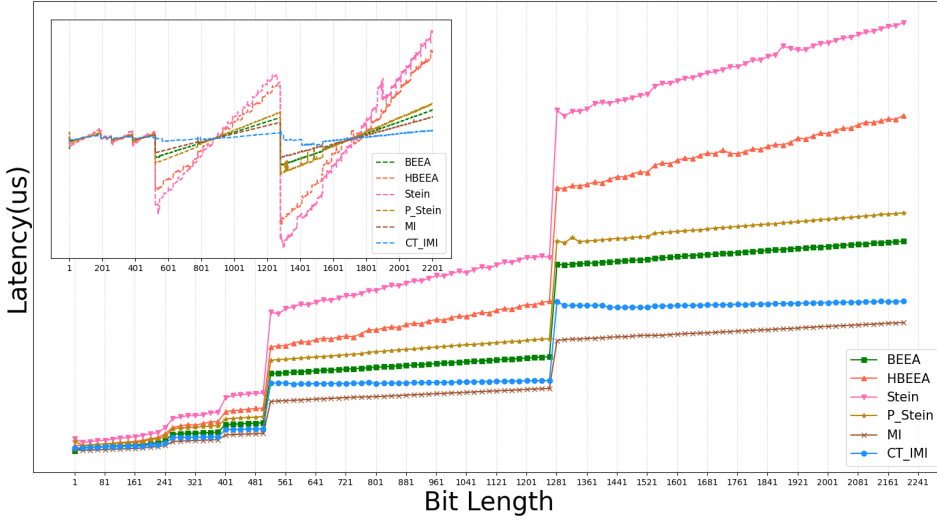


Fig. 9. Comparison of latency between constant and non-constant time modular inverse algorithm for different bit length numbers.

Table 5. Comparison of Average Operating Clock Cycles of Constant Time Modular Inverse Algorithms

	192 bits	224 bits	256 bits	384 bits	521 bits	1,279 bits	2,203 bits
FLT	0.34M	0.44M	0.56M	1.22M	2.73M	27.13M	138.94M
BOS	0.33M	0.43M	0.51M	0.95M	1.53M	4.13M	8.85M
BY	0.32M	0.41M	0.51M	0.87M	1.23M	3.65M	7.55M
hdBY	0.30M	0.38M	0.49M	0.75M	1.02M	3.02M	6.22M
SICT-MI	0.29M	0.34M	0.40M	0.62M	0.86M	2.39M	5.21M
CT-IMI	0.20M	0.23M	0.27M	0.41M	0.58M	1.60M	3.31M

the other two. This is mainly due to the multiplication of large integers. So we can conclude that FLT is not suitable when processing long-bit length numbers.

6.3 Hardware Results and Comparisons

For each prime, even though the iteration architecture is universal, the Montgomery and modular reduction architectures are different in order to pursue highly efficient hardware implementation. We design the universal iteration architecture and the reduction architecture for P_{sm2} , which is widely used in ECC. Table 6 shows the experimental results, and two **Area&Time (AT)** products are used to evaluate the performance. For CT-IMI, the input numbers are integers and CT-CMMI requires inputs in Montgomery form. Modular reduction is simpler than Montgomery reduction, which results in CT-CMMI costing fewer hardware resources than CT-IMI. On Virtex-7, Spartan-6, and Virtex-5 FPGA, each slice contains 4 LUTs and 8 FFs. In contrast, the slice on Ultrascale consists of 8 LUTs and 16 FFs. This means that Ultrascale requires fewer slices to implement the same design, due to the increased number of LUTs and FFs per slice. To the best of the authors' knowledge, all

Table 6. Comparison of Hardware Implementations of 256-bit Modular Inversion

Ref.	Technology	Freq. (MHz)	LUT	FF	Cycle	Slices	Latency (μ s)	AT^I (LUT $\times$ ms)	AT^{II} (Slice $\times$ ms)
CT-IMI	Ultrascale	151.4	4.2k	1.8k	295	712	1.95	8.19	1.39
	Virtex-7	115.9	4.1k	1.8k		1132	2.56	10.50	2.90
CT-CMMI	Ultrascale	152.7	2.7k	1.6k	285	459	1.87	5.05	0.86
	Virtex-7	116.3	2.7k	1.6k		854	2.45	6.62	2.09
Hossain [11]	Virtex-7	146.38	5.8k	1.3k	340	1480	2.32	13.47	3.43
Deshpande [8]	Ultrascale	190.4	unknown	0.8k	8949	915	47.00	unknown	43.01
Murat [26]	Spartan-6	231.87	1.1k	0.5k	5120	unknown	22.08	24.29	unknown
Mrabet [25]	Virtex-5	129	2.3k	1.1k	1024	592	7.93	18.24	4.70

AT^I LUT cost $\times$ latency for one time operation AT^{II} Slice cost $\times$ latency for one time operation.

the latest research results, which are related to the implementation of modular inversion for 256 bits prime on FPGA, have been presented in Table 6.

On Virtex-7, our CT-IMI has a latency of 2.56 μ s and requires 4.1k LUTs, 1.8k FFs and 1,132 slices. Our CT-CMMI has a latency of 2.45 μ s and requires 2.7k LUTs, 1.6k FFs and 854 slices. Hossain's design has the lowest latency of 2.32 μ s, but it requires 5.8k LUTs, 1.3k FFs and 1,480 slices. Compared to Hossain's design, our CT-IMI saves around 27.6% LUTs and 23.5% slices, while our CT-CMMI saves around 53.4% LUTs and 42.3% slices. Murat's design has the highest maximum frequency and uses the least hardware resources with 1.1k LUTs and 0.5k FFs, making it suitable for lightweight applications when latency is not critical. However, the clock cycles in our designs are far less than those in Murat's design, and our latency is around five times lower. Mrabetm's design costs 2.3k LUTs, 1.1k FFs and 592 slices, with a latency of 7.93 μ s. The area costs are lower than ours, but our CT-IMI and CT-CMMI designs save around 41.3% and 43.3% time, respectively.

On Ultrascale, CT-IMI has a latency of 1.95 μ s and requires 4.2k LUTs, 1.8k FFs, and 712 slices. CT-CMMI has a latency of 1.87 μ s and requires 2.7k LUTs, 1.6k FFs, and 459 slices. Deshpande proposed a universal hardware architecture for different bit length numbers. For 256 bits inputs, his design also has constant time property and costs 915 slices. Our CT-IMI saves 22.2% Slices and CT-CMMI saves 49.8% Slices. The latency of his design is around 23.5 times ours.

Figure 10 shows the AT^I and AT^{II} for each design. Compared with other designs, our design balances area and time. The performance of A&T reaches an optimal level. When considering the performance of A&T, Hossain achieved a better result than Deshpande, Murat and Mrabet. However, Hossain's design took 1.28 times AT^I , 1.18 times AT^{II} of our CT-IMI, and 2.03 times AT^I , 1.64 times AT^{II} of our CT-CMMI on Virtex-7, respectively. The hardware results show that our designs have better performance than other available designs in Table 6. Besides, our architectures have advantages due to the constant time feature and the CT-CMMI can directly compute with Montgomery domain numbers.

Although our designs exhibit high performance and are well-suited for practical applications, there is still some room for further optimizations. A possible way is inserting some registers between the cascaded MUXs in the whole iterative structure. The pipeline can break long timing

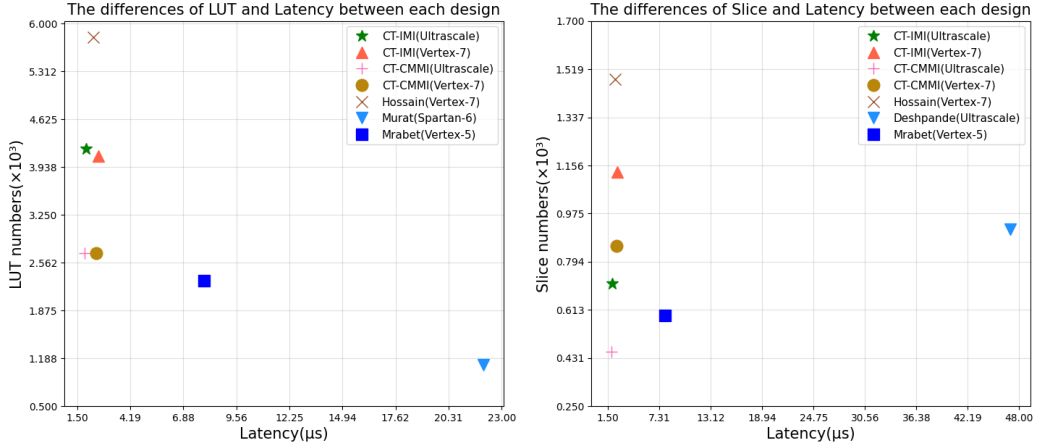


Fig. 10. Comparison of AT^I and AT^{II} for different hardware implementations.

paths, which can also increase the maximum operating frequency of the overall circuit. In addition, there are some arithmetic units in our design, such as adders and subtractors. Considering the large scale of operands, such as 256-bit, the latency introduced by carry chains is considerable. Optimizations for these arithmetic units, such as parallel prefix adders, carry-lookahead adders, can help to mitigate the latency. However, it is worth noting that these methods may also lead to increased resource consumption. So further exploration needs more careful tradeoffs between resources and latency.

7 Conclusion

This article mainly proposes two modular inversion algorithms, CT-IMI and CT-CMMI. Both of them have the constant time feature, which allows them to resist the timing attack. In software analysis, we make a comparison with other non-constant algorithms and show the significant differences in latency when the bit length increases. Therefore, the constant time feature is necessary for modular inversion algorithms when considering the safety against SCAs. Besides, CT-IMI is a classic design, which processes modular inversion on the integer domain. In comparison, CT-CMMI is a cross-domain design, which uses the Montgomery numbers as inputs and directly computes the inversion in the integer domain. Compared to CT-IMI, CT-CMMI avoids the conservation steps for some specific applications. In order to pursue high efficiency, we use a new method to analyze the boundary of iteration rounds. After sampling around 10^7 random numbers and counting the iteration rounds, the distribution is very likely to be a Gaussian distribution. We use the maximum iteration round that occurs in experiments as the constant iteration round, fitting the Gaussian distribution and showing the probability of the correctness of using this value. Finally, we propose our hardware architectures on FPGA, consisting of a new hardware iteration architecture that is universal for any prime, and a specific Montgomery and modular reduction for P_{sm2} . This is also the first time that the cross-domain modular inversion algorithm has been implemented on hardware to the best of our knowledge. Compared with the existing literature, both the clock cycles in software and AT product in hardware of our designs reach the optimal. We believe that CT-IMI and CT-CMMI will be used in high-efficiency cryptosystem processors in the future.

Acknowledgments

The authors would like to thank the anonymous reviewers for their valuable comments.

References

- [1] Daniel J. Bernstein and Bo-Yin Yang. 2019. Fast Constant-time GCD computation and modular inversion. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2019, 3 (2019), 340–398.
- [2] Karim Bigou and Arnaud Tisserand. 2013. Improving modular inversion in RNS using the plus-minus method. In *Proceedings of the Cryptographic Hardware and Embedded Systems—CHES 2013: 15th International Workshop, Santa Barbara, CA, USA*. Springer, 233–249.
- [3] Karim Bigou and Arnaud Tisserand. 2016. Binary-Ternary plus-minus modular inversion in RNS. *IEEE Transactions on Computers* 65, 11 (2016), 3495–3501.
- [4] Joppe W. Bos. 2014. Constant time modular inversion. *Journal of Cryptographic Engineering* 4 (2014), 275–281.
- [5] Lily Chen, Dustin Moody, Andrew Regenscheid, and Karen Randall. 2019. *Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters*. Technical Report. National Institute of Standards and Technology.
- [6] Piljoo Choi, Jeong-Taek Kong, and Dong Kyue Kim. 2015. Analysis of hardware modular inversion modules for elliptic curve cryptography. In *Proceedings of the 2015 International SoC Design Conference (ISOCC)*. 313–314.
- [7] Richard E. Crandall. 1992. Method and Apparatus for Public Key Exchange in a Cryptographic System. US Patent 5,159,632.
- [8] Sanjay Deshpande, Santos Merino del Pozo, Victor Mateu, Marc Manzano, Najwa Aaraj, and Jakub Szefer. 2021. Modular inverse for integers using fast constant time GCD algorithm and its applications. In *Proceedings of the 2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. 122–129.
- [9] W. Diffie and M. Hellman. 1976. New directions in cryptography. *IEEE Transactions on Information Theory* 22, 6 (1976), 644–654.
- [10] Laszlo Hars. 2006. Modular inverse algorithms without multiplications for cryptographic applications. *EURASIP Journal on Embedded Systems* 2006, 1 (2006), 1–13.
- [11] Md Selim Hossain and Yinan Kong. 2015. High-Performance FPGA implementation of modular inversion over F_{256} for elliptic curve cryptography. In *Proceedings of the 2015 IEEE International Conference on Data Science and Data Intensive Systems*. 169–174.
- [12] Jin Hu and Yongbin Li. 2022. An improved modular inversion algorithm and its hardware implementation. *Journal of Hunan University (Natural Sciences)* 49, 02 (2022), 101–105.
- [13] Yaoan Jin and Atsuko Miyaji. 2023. Short-Iteration constant-time GCD and modular inversion. In *Proceedings of the Smart Card Research and Advanced Applications: 21st International Conference, CARDIS 2022, November 7–9, 2022, Revised Selected Papers*. Springer-Verlag, 82–99.
- [14] B. S. Kaliski. 1995. The montgomery inverse and its applications. *IEEE Transactions on Computers* 44, 8 (1995), 1064–1065.
- [15] Donald E. Knuth. 1998. *The Art of Computer Programming: Seminumerical Algorithms* (3rd ed.). Addison-Wesley Publishing Company, Massachusetts.
- [16] Neal Koblitz. 1987. Elliptic curve cryptosystems. *Mathematics of Computation* 48, 177 (1987), 203–209.
- [17] Edmund Landau. 1999. *Elementary Number Theory* (2nd. ed.). AMS Chelsea Pub., Providence, RI.
- [18] The GNU Multiple Precision Arithmetic Library. 2021. GMP library. Retrieved February 18, 2023 from <https://gmplib.org/>
- [19] Zhining Lim, Braden J. Phillips, and Michael Liebelt. 2009. Elliptic curve digital signature algorithm over $GF(p)$ on a residue number system enabled microprocessor. In *Proceedings of the TENCON 2009—2009 IEEE Region 10 Conference*. 1–6.
- [20] Chen Lin and Yi Wang. 2022. Implementation of cache timing attack based on present algorithm. In *Proceedings of the 2022 8th Annual International Conference on Network and Information Systems for Computers (ICNISC)*. IEEE, 32–35.
- [21] Róbert Lórencz. 2003. New algorithm for classical modular inverse. In *Proceedings of the Cryptographic Hardware and Embedded Systems—CHES 2002 (Lecture Notes in Computer Science)*. Springer, 57–70.
- [22] M. Donald MacLaren. 1970. The art of computer programming. volume 2: Seminumerical algorithms (Donald E. Knuth). *SIAM Review* 12, 2 (1970), 306–308.
- [23] Victor S. Miller. 1986. Use of elliptic curves in cryptography. In *Proceedings of the Advances in Cryptology—CRYPTO’85*, Hugh C. Williams (Ed.). Springer, Berlin, 417–426.
- [24] Peter L. Montgomery. 1985. Modular multiplication without trial division. *Mathematics of Computation* 44, 170 (1985), 519–521.
- [25] Amine Mrabet, Nadia El-Mrabet, Belgacem Bouallegue, Sihem Mesnager, and Mohsen Machhout. 2017. An efficient and scalable modular inversion/division for public key cryptosystems. In *Proceedings of the 2017 International Conference on Engineering and MIS (ICEMIS)*. 1–6.
- [26] Ertugrul Murat, Süleyman Kardaş, and Erkey Savaş. 2011. Scalable and efficient FPGA implementation of montgomery inversion. In *Proceedings of the 2011 Workshop on Lightweight Security and Privacy: Devices, Protocols, and Applications*. 61–68.
- [27] National Standardization Technical Committee of Information Security of China. 2017. *Information Technology - SM2 Elliptic Curve Public Key Cryptographic Algorithm - Part 5: Parameter Definition*. GB/T 32918.5-2017.

- [28] William M. Daley and Raymond G. Kammer. 2000. Digital signature standard (DSS). *National Institute of Standards*. 1–74. Retrieved from <https://apps.dtic.mil/sti/html/tr/ADA399987/>
- [29] Pascal Sasdrich and Tim Güneysu. 2015. Implementing curve25519 for side-channel-protected elliptic curve cryptography. *ACM Transactions on Reconfigurable Technology and Systems* 9, 1 (2015), 1–15.
- [30] E. Savas and C. K. Koc. 2000. The montgomery modular inverse-revisited. *IEEE Transactions on Computers* 49, 7 (2000), 763–766.
- [31] Jerome A. Solinas. 2011. *Generalized Mersenne Prime*. Springer US, Boston, MA, 509–510.
- [32] Burcu Sönmez, Ahmet Ali Sarıkaya, and Şerif Bahtiyar. 2019. Machine learning based side channel selection for time-driven cache attacks on AES. In *Proceedings of the 2019 4th International Conference on Computer Science and Engineering (UBMK)*. 1–5.
- [33] Keke Wu, Huiyun Li, Tingding Chen, and Fengqi Yu. 2009. Simple power analysis on elliptic curve cryptosystems and countermeasures: practical work. In *Proceedings of the 2009 2nd International Symposium on Electronic Commerce and Security*. 21–24.
- [34] P. Wuille and G. Maxwell. 2021. Roconnor-Blockstream: Safegcd-Bounds. Retrieved from <https://github.com/sipa/safegcd-bounds>
- [35] Sen Xu, Haihua Gu, Lingyun Wang, Zheng Guo, Junrong Liu, Xiangjun Lu, and Dawu Gu. 2017. Efficient and constant time modular inversions over prime fields. In *Proceedings of the 2017 13th International Conference on Computational Intelligence and Security (CIS)*. 524–528.
- [36] Xiaodong Yan and Shuguo Li. 2007. Modified modular inversion algorithm for VLSI implementation. In *Proceedings of the 2007 7th International Conference on ASIC*. 90–93.
- [37] Tao Zhou, Xingjun Wu, Guoqiang Bai, and Hongyi Chen. 2002. New algorithm and fast VLSI implementation for modular inversion in galois field $GF(p)$. In *Proceedings of the IEEE 2002 International Conference on Communications, Circuits and Systems and West Sino Expositions*. 1491–1495.

Received 31 August 2023; revised 26 April 2025; accepted 4 October 2025